

90-Day DevOps Journey

Overall Roadmap

Days	Phase
01-07	01-linux
08-14	02-bash
15-21	03-networking
22-28	04-git
29-40	05-docker
41-47	06-jenkins
48-54	07-github-actions
55-67	08-kubernetes
68-74	09-argocd
75-80	10-ansible
81-86	11-terraform
87-88	12-aws
89-90	capstone

This gives more time to Docker and Kubernetes, because they require substantially more hands-on practice.

Phase 1 — Linux

Days 01-07

Directory:

```
01-linux/  
├── setup-server.sh  
└── permissions-lab.md
```

Day 01 — Linux Fundamentals

Topics - What is Linux? - Linux kernel - Linux distributions - Ubuntu - Linux server vs desktop - Shell vs terminal - Root user - sudo

Hands-on

Run and understand:

```
uname -a
hostname
whoami
id
uptime
date
ls
pwd
```

Output

Create:

```
01-linux/day-01-notes.md
```

Commit

```
day-01: learn linux fundamentals
```

Day 02 — Linux Filesystem

Topics

Learn:

```
/
├── bin
├── boot
├── dev
├── etc
├── home
├── opt
├── tmp
├── usr
└── var
```

Understand: - `/etc` - `/var` - `/home` - `/opt` - `/tmp` - `/usr`

Hands-on

Create:

```
/opt/myapp/
├── config/
├── data/
├── logs/
└── backups/
```

Practice:

```
pwd
ls
cd
mkdir
touch
find
tree
```

Output

```
01-linux/filesystem-lab.md
```

Commit

```
day-02: practice linux filesystem
```

Day 03 — Files and Directories

Topics - File creation - Copy - Move - Delete - Wildcards - Hidden files - Symbolic links - Hard links

Hands-on

Practice:

```
touch  
cp  
mv  
rm  
ln
```

Create symbolic and hard links and investigate the difference.

Output

```
01-linux/file-operations.md
```

Commit

```
day-03: practice linux file operations
```

Day 04 — Linux Permissions

Topics - Owner - Group - Others - Read - Write - Execute - `chmod` - `chown` - `umask`

Understand: - `755` - `750` - `700` - `644` - `640` - `600`

Hands-on

Create files with different permissions and test access.

Output

```
01-linux/permissions-lab.md
```

Commit

```
day-04: practice linux permissions
```

Day 05 — Users and Groups

Topics - `/etc/passwd` - `/etc/group` - Users - Groups - `sudo` - `useradd` - `usermod` - `groupadd` - `passwd`

Hands-on

Create: - `devops` - `developer` - `deployment`
and configure groups.

Output

```
01-linux/users-lab.md
```

Commit

```
day-05: manage linux users and groups
```

Day 06 — Processes + Packages + Services

Topics - Processes - PID - `ps` - `top` - `htop` - `kill` - `systemctl` - `journalctl` - `apt`

Hands-on

Install:

```
git
curl
wget
tree
htop
```

Investigate running services.

Output

```
01-linux/process-service-lab.md
```

Commit

```
day-06: practice processes packages and services
```

Day 07 — Linux Server Automation

Project

Create:

```
01-linux/setup-server.sh
```

The script should: - Update packages - Install required tools - Create users - Create groups - Create application directories - Set permissions - Display server information

Output

```
01-linux/setup-server.sh
```

Commit

```
day-07: complete linux server setup automation
```

Phase 2 — Bash

Days 08-14

Directory:

```
02-bash/
├─ system-info.sh
└─ backup.sh
```

Day 08 — Bash Fundamentals

Topics - Shebang - Variables - `echo` - Command substitution - Environment variables - Exit codes

Hands-on

Create:

```
02-bash/system-info.sh
```

Output:

```
Hostname:
OS:
Kernel:
CPU:
Memory:
Disk:
Uptime:
```

Commit

```
day-08: create system info script
```

Day 09 — Conditions

Topics - `if` - `elif` - `else` - `test` - `[ ]` - `[[ ]]` - `case`

Hands-on

Create a script that checks: - Does user exist? - Does file exist? - Is service running? - Is disk usage above threshold?

Output

```
02-bash/system-check.sh
```

Commit

```
day-09: practice bash conditions
```

Day 10 — Loops

Topics - `for` - `while` - `until`

Hands-on

Create a script that checks multiple servers or directories.

Output

```
02-bash/loop-lab.sh
```

Commit

```
day-10: practice bash loops
```

Day 11 — Functions + Arguments

Topics - Functions - `$1` - `$2` - `$@` - `$#` - `$?`

Hands-on

Create:

```
02-bash/service-manager.sh
```

Example:

```
./service-manager.sh nginx status  
./service-manager.sh nginx restart
```

Commit

```
day-11: build bash service manager
```

Day 12 — Bash Error Handling

Topics - Exit status - `set -e` - `set -u` - `set -o pipefail` - Logging - Error handling

Hands-on

Improve previous scripts with proper error handling.

Output

```
02-bash/error-handling.md
```

Commit

```
day-12: add bash error handling
```

Day 13 — Backup Automation

Topics - `tar` - Compression - Timestamps - Backup retention

Hands-on

Create:

```
02-bash/backup.sh
```

It should:

```
Source  
↓  
tar.gz  
↓  
backup directory  
↓  
delete old backups
```

Commit

```
day-13: build automated backup script
```

Day 14 — Bash Automation Project

Combine your scripts.

Project

```
02-bash/
├─ system-info.sh
├─ system-check.sh
├─ service-manager.sh
├─ backup.sh
└─ README.md
```

Commit

```
day-14: complete bash automation toolkit
```

Phase 3 — Networking + SSH

Days 15-21

Directory:

```
03-networking/
├─ networking-lab.md
└─ ssh-lab.md
```

Day 15 — Networking Fundamentals

Topics - IP address - MAC address - IPv4 - IPv6 - Private IP - Public IP

Hands-on

Investigate your machine:

```
ip addr
ip route
```

Output

```
03-networking/networking-lab.md
```

Commit

```
day-15: learn networking fundamentals
```

Day 16 — TCP/IP

Topics - TCP - UDP - Ports - Client/server - OSI model - TCP/IP model

Hands-on

Investigate:

```
ss -tulpn
```

Find running services and their ports.

Commit

```
day-16: practice tcp ip and ports
```

Day 17 — DNS

Topics - DNS - Domain - Resolver - A record - AAAA - CNAME - Nameserver

Hands-on

Use:

```
dig
nslookup
host
```

Commit

```
day-17: practice dns troubleshooting
```

Day 18 — HTTP/HTTPS

Topics - HTTP - HTTPS - Request - Response - Headers - Status codes - TLS

Hands-on

Use:

```
curl
```

Investigate:

```
200
301
302
400
401
403
404
500
```

Commit

```
day-18: investigate http and https
```

Day 19 — SSH

Topics - SSH - Public/private keys - `authorized_keys` - SSH config

Hands-on

Generate an SSH key:

```
ssh-keygen
```

Configure SSH access.

Output

```
03-networking/ssh-lab.md
```

Commit

```
day-19: configure ssh key authentication
```


Day 20 — Networking Troubleshooting

Topics - `ping` - `curl` - `ss` - `dig` - `traceroute`

Hands-on

Create a troubleshooting scenario:

```
Application cannot connect
      ↓
DNS?
      ↓
IP?
      ↓
Port?
      ↓
Firewall?
      ↓
Service?
```

Commit

```
day-20: build networking troubleshooting lab
```

Day 21 — Networking Project

Create a complete troubleshooting guide.

Output

```
03-networking/
├─ networking-lab.md
├─ ssh-lab.md
└─ troubleshooting.md
```

Commit

```
day-21: complete networking and ssh labs
```

Phase 4 — Git + GitHub

Days 22-28

Directory:

```
04-git/
├─ git-workflow.md
└─ git-recovery.md
```

Day 22 — Git Fundamentals

Learn:

```
git init
git status
git add
git commit
git log
git diff
```

Commit

```
day-22: learn git fundamentals
```

Day 23 — Branching

Learn:

```
git switch  
git branch  
git merge
```

Hands-on

Create:

```
feature/linux-monitoring
```

Commit

```
day-23: practice git branching
```

Day 24 — GitHub

Learn: - Remote - Push - Pull - Fetch - Clone - SSH authentication

Commit

```
day-24: connect repository to github
```

Day 25 — Merge Conflicts

Intentionally create a conflict.

Resolve it manually.

Output

```
04-git/git-workflow.md
```

Commit

```
day-25: practice merge conflict resolution
```

Day 26 — Rebase + Squash

Learn:

```
git rebase  
git rebase -i
```

Practice squashing commits.

Commit

```
day-26: practice git rebase and squash
```

Day 27 — Git Recovery

Learn:

```
git stash
git restore
git reset
git revert
git reflog
```

Output

```
04-git/git-recovery.md
```

Commit

```
day-27: practice git recovery techniques
```

Day 28 — Professional Git Workflow

Build:

```
feature
↓
commit
↓
push
↓
PR
↓
review
↓
rebase
↓
merge
↓
tag
```

Output

```
04-git/git-workflow.md
```

Commit

```
day-28: complete professional git workflow
```

Phase 5 — Docker

Days 29-40

Directory:

```
05-docker/
├─ Dockerfile
└─ docker-compose.yml
```

Day 29 — Containers

Learn: - Container - Image - Registry - Runtime

Practice:

```
docker run
docker ps
docker stop
docker rm
```

Commit

```
day-29: run first docker containers
```

Day 30 — Docker Images

Practice:

```
docker pull
docker images
docker inspect
docker rmi
```

Commit

```
day-30: practice docker images
```

Day 31 — Dockerfile

Learn:

```
FROM
RUN
COPY
WORKDIR
ENV
EXPOSE
CMD
ENTRYPOINT
```

Hands-on

Containerize a simple application.

Output

```
05-docker/Dockerfile
```

Commit

```
day-31: create application dockerfile
```

Day 32 — Docker Build

Learn: - Build context - Layers - Cache

Practice:

```
docker build
```

Commit

```
day-32: build docker image
```

Day 33 — Docker Networking

Learn: - Bridge - Host - Container networking - Port mapping

Hands-on

Connect two containers.

Commit

```
day-33: practice docker networking
```

Day 34 — Volumes

Learn: - Bind mounts - Named volumes - Persistent data

Hands-on

Run PostgreSQL with persistent storage.

Commit

```
day-34: practice docker volumes
```

Day 35 — Environment Variables

Learn: - `ENV` - `--env` - `.env`

Hands-on

Move application configuration into environment variables.

Commit

```
day-35: configure docker environment variables
```

Day 36 — Docker Compose

Learn: - Services - Networks - Volumes - Dependencies

Output

```
05-docker/docker-compose.yml
```

Commit

```
day-36: create docker compose stack
```

Day 37 — Multi-Container Application

Build:

```
Nginx
  ↓
Backend
  ↓
PostgreSQL
```

Commit

day-37: build multi container application

Day 38 — Docker Healthchecks

Add: - `healthcheck` - `depends_on` - `restart`

Commit

day-38: add docker healthchecks

Day 39 — Docker Image Optimization

Learn: - `.dockerignore` - Layer optimization - Multi-stage builds - Smaller images

Commit

day-39: optimize docker image

Day 40 — Docker Project

Finalize the application.

Output

```
05-docker/  
├── Dockerfile  
├── docker-compose.yml  
└── README.md
```

Commit

day-40: complete docker deployment project

Phase 6 — Jenkins

Days 41-47

Directory:

```
06-jenkins/  
├── Jenkinsfile  
└── docker-compose.yml
```

Day 41 — CI/CD Concepts

Learn: - CI - CD - Pipeline - Job - Stage - Artifact

Commit

day-41: learn cicd fundamentals

Day 42 — Jenkins Setup

Run Jenkins using Docker.

Output

```
06-jenkins/docker-compose.yml
```

Commit

```
day-42: setup jenkins
```

Day 43 — Jenkins Job

Create a basic job:

```
Checkout
↓
Build
↓
Test
```

Commit

```
day-43: create first jenkins job
```

Day 44 — Jenkinsfile

Create:

```
06-jenkins/Jenkinsfile
```

Pipeline:

```
Checkout
↓
Test
↓
Build
```

Commit

```
day-44: create jenkins pipeline
```

Day 45 — Docker + Jenkins

Pipeline:

```
Git
↓
Jenkins
↓
Docker Build
```

Commit

```
day-45: integrate docker with jenkins
```

Day 46 — Jenkins Pipeline Improvements

Add: - Environment variables - Post actions - Failure handling

Commit

day-46: improve jenkins pipeline

Day 47 — Jenkins Project

Complete:

```
GitHub
↓
Jenkins
↓
Test
↓
Docker Build
```

Commit

day-47: complete jenkins ci project

Phase 7 — GitHub Actions

Days 48-54

Directory:

```
07-github-actions/
└─ ci.yml
```

Day 48 — GitHub Actions Fundamentals

Learn: - Workflow - Job - Step - Runner - Trigger

Commit

day-48: learn github actions

Day 49 — First Workflow

Create:

```
07-github-actions/ci.yml
```

Trigger:

```
on:
  push:
  pull_request:
```

Commit

day-49: create first github actions workflow

Day 50 — Automated Testing

Add:


```
Checkout
↓
Install dependencies
↓
Run tests
```

Commit

```
day-50: add automated tests to ci
```

Day 51 — Linting

Add:

```
Lint
↓
Test
```

Commit

```
day-51: add linting to ci
```

Day 52 — Docker Build

Add:

```
Test
↓
Docker Build
```

Commit

```
day-52: build docker image in ci
```

Day 53 — Container Registry

Publish image to GHCR.

Commit

```
day-53: publish image from github actions
```

Day 54 — CI Project

Final pipeline:

```
Push
↓
GitHub Actions
↓
Lint
↓
Test
↓
Docker Build
↓
GHCR
```

Commit

day-54: complete github actions ci pipeline

Phase 8 — Kubernetes

Days 55-67

Directory:

```
08-kubernetes/  
├─ deployment.yaml  
├─ service.yaml  
├─ configmap.yaml  
└─ secret.yaml
```

Day 55 — Kubernetes Architecture

Learn: - Cluster - Control plane - Node - API server - Scheduler - Controller Manager - etcd - kubelet

Commit

day-55: learn kubernetes architecture

Day 56 — Local Cluster

Use:

kind

or:

minikube

Commit

day-56: create local kubernetes cluster

Day 57 — Pods

Learn:

```
kubectl get pods  
kubectl describe pod  
kubectl logs
```

Deploy your first pod.

Commit

day-57: deploy first kubernetes pod

Day 58 — Deployments

Learn: - Deployment - ReplicaSet - Replicas

Output

```
08-kubernetes/deployment.yaml
```

Commit

```
day-58: create kubernetes deployment
```

Day 59 — Services

Learn: - ClusterIP - NodePort - LoadBalancer

Output

```
08-kubernetes/service.yaml
```

Commit

```
day-59: expose application with kubernetes service
```

Day 60 — ConfigMaps + Secrets

Create:

```
configmap.yaml  
secret.yaml
```

Commit

```
day-60: configure kubernetes configmaps and secrets
```

Day 61 — Health Probes

Learn: - Liveness - Readiness - Startup

Commit

```
day-61: configure kubernetes health probes
```

Day 62 — Resource Management

Learn: - CPU requests - Memory requests - Limits

Commit

```
day-62: configure kubernetes resources
```

Day 63 — Rolling Updates

Deploy v1.

Then deploy v2.

Observe:

```
v1  
↓  
v2
```

Commit

```
day-63: practice kubernetes rolling updates
```

Day 64 — Rollbacks

Intentionally deploy a bad version.

Recover with:

```
kubectl rollout undo
```

Commit

```
day-64: practice kubernetes rollback
```

Day 65 — HPA

Learn: - Horizontal Pod Autoscaler - Metrics - Scaling

Commit

```
day-65: configure kubernetes autoscaling
```

Day 66 — Kubernetes Debugging

Practice:

```
kubectl logs  
kubectl describe  
kubectl get events  
kubectl exec
```

Create a broken deployment and fix it.

Commit

```
day-66: practice kubernetes troubleshooting
```

Day 67 — Kubernetes Project

Deploy your Docker application completely.

- Deployment
- Service
- ConfigMap
- Secret
- Probes
- Resources
- HPA

Commit

```
day-67: complete kubernetes deployment project
```

Phase 9 — Helm + ArgoCD + GitOps

Days 68-74

Directory:

```
09-argocd/  
├─ application.yaml  
└─ helm-chart/
```

Day 68 — Helm Fundamentals

Learn: - Chart - Template - Values - Release

Create:

```
09-argocd/helm-chart/
```

Commit

```
day-68: create first helm chart
```

Day 69 — Helm Templates

Learn:

```
Chart.yaml  
values.yaml  
templates/
```

Convert your Kubernetes manifests into Helm templates.

Commit

```
day-69: convert kubernetes manifests to helm
```

Day 70 — Helm Deployment

Deploy with:

```
helm install  
helm upgrade  
helm list  
helm uninstall
```

Commit

```
day-70: deploy application with helm
```

Day 71 — GitOps Concepts

Understand:

```
Desired state  
  ↓  
Git  
  ↓  
Controller
```

↓
Cluster

Commit

day-71: learn gitops principles

Day 72 — ArgoCD

Install ArgoCD locally.

Commit

day-72: setup argocd

Day 73 — ArgoCD Application

Create:

09-argocd/application.yaml

Connect Git repository → Kubernetes.

Commit

day-73: deploy application with argocd

Day 74 — GitOps Automation

Enable: - Auto sync - Self healing - Git-based rollback

Final flow:

```
GitHub
  ↓
Git change
  ↓
ArgoCD
  ↓
Kubernetes
```

Commit

day-74: complete gitops deployment

Phase 10 — Ansible

Days 75-80

Directory:

```
10-ansible/
├─ inventory/
├─ playbook.yml
├─ roles/
└─ templates/
```

Day 75 — Ansible Architecture

Learn: - Controller - Managed node - Inventory - Module - Playbook - Task

Commit

```
day-75: learn ansible architecture
```

Day 76 — Inventory + Ad-Hoc Commands

Create:

```
10-ansible/inventory/hosts.ini
```

Practice:

```
ansible all -m ping
ansible all -m shell -a "uptime"
```

Commit

```
day-76: create ansible inventory
```

Day 77 — Playbooks

Create:

```
10-ansible/playbook.yml
```

Automate:

```
Install packages
↓
Create user
↓
Create directories
↓
Configure service
```

Commit

```
day-77: create ansible configuration playbook
```

Day 78 — Variables + Templates

Learn: - Variables - Facts - Jinja2

Create:

```
10-ansible/templates/
```

Commit

```
day-78: practice ansible variables and templates
```

Day 79 — Roles

Create:

```
10-ansible/roles/  
└─ nginx/
```

Learn: - tasks - handlers - templates - defaults - vars

Commit

```
day-79: create reusable ansible role
```

Day 80 — Ansible Project

Automate a complete server:

```
Ansible  
  ↓  
Install software  
  ↓  
Create users  
  ↓  
Configure application  
  ↓  
Configure Nginx  
  ↓  
Start services
```

Run the playbook twice and verify idempotency.

Commit

```
day-80: complete ansible server automation
```

Phase 11 — Terraform

Days 81-86

Directory:

```
11-terraform/  
├─ main.tf  
├─ variables.tf  
└─ outputs.tf
```

Day 81 — Terraform Fundamentals

Learn: - IaC - Provider - Resource - State - Configuration

Practice:

```
terraform init  
terraform fmt  
terraform validate  
terraform plan
```

Commit

```
day-81: learn terraform fundamentals
```


Day 82 — Terraform Resources

Create your first resource.

Understand: - resource - provider - dependency

Commit

```
day-82: create terraform resources
```

Day 83 — Variables

Create:

```
11-terraform/variables.tf
```

Learn: - Input variables - Defaults - Types

Commit

```
day-83: add terraform variables
```

Day 84 — Outputs

Create:

```
11-terraform/outputs.tf
```

Learn how to expose resource information.

Commit

```
day-84: add terraform outputs
```

Day 85 — Terraform Lifecycle

Practice:

```
terraform plan
terraform apply
terraform destroy
```

Understand:

```
Configuration
↓
Plan
↓
Apply
↓
State
```

Commit

```
day-85: practice terraform lifecycle
```

Day 86 — Terraform Project

Build a small infrastructure project.

Document:

```
main.tf
variables.tf
outputs.tf
README.md
```

Commit

```
day-86: complete terraform infrastructure project
```

Phase 12 — AWS

Days 87-88

Directory:

```
12-aws/
└─ architecture.md
```

Day 87 — AWS Infrastructure

Study and practice: - IAM - VPC - Subnet - Route table - Internet Gateway - Security Group - EC2 - S3 - ECR

Hands-on

Map your previous Terraform concepts to AWS architecture.

Create:

```
12-aws/architecture.md
```

Commit

```
day-87: document aws infrastructure architecture
```

Day 88 — AWS DevOps Architecture

Design:

```
GitHub
  ↓
GitHub Actions
  ↓
ECR
  ↓
EKS
  ↓
Application
```

Understand: - ECR - EKS - IAM roles - VPC - Security groups - CloudWatch

Output

Update:

```
12-aws/architecture.md
```

Commit

day-88: design aws devops architecture

Phase 13 — Final Capstone

Days 89-90

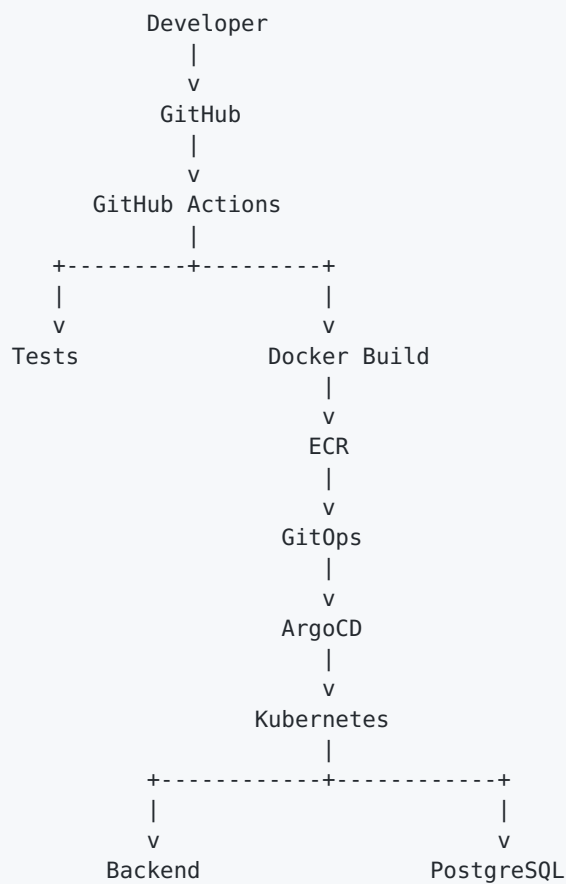
Directory:

```
capstone/  
├── README.md  
└── architecture.md
```

Day 89 — Build the DevOps Platform

Bring everything together.

Your target architecture:



Infrastructure:

```
Terraform  
↓  
AWS
```

Configuration:

```
Ansible
  ↓
Server configuration
```

Hands-on

Verify that you can explain and demonstrate every major part.

Output

```
capstone/architecture.md
```

Commit

```
day-89: integrate complete devops platform
```

Day 90 — Final Capstone + Documentation

This is not another learning day.

This is your demonstration and cleanup day.

Tasks

1. Clean repository

Check:

```
README.md
progress.md
broke-it-fixed-it.md
```

2. Verify projects

```
01-linux          ✓
02-bash           ✓
03-networking     ✓
04-git            ✓
05-docker         ✓
06-jenkins        ✓
07-github-actions ✓
08-kubernetes     ✓
09-argocd         ✓
10-ansible        ✓
11-terraform      ✓
12-aws            ✓
capstone          ✓
```

3. Write final architecture

```
capstone/architecture.md
```

4. Write final README

Include: - Project overview - Architecture - Technologies - CI/CD flow - Kubernetes flow - GitOps flow - Infrastructure - Security - Lessons learned

5. Final Git commit

```
git add .
git commit -m "day-90: complete 90-day devops journey"
git push
```

Your 90-Day Commit History

By the end, your Git history should look roughly like:

```
day-90: complete 90-day devops journey
day-89: integrate complete devops platform
day-88: design aws devops architecture
day-87: document aws infrastructure architecture
day-86: complete terraform infrastructure project
day-85: practice terraform lifecycle
...
day-03: practice linux file operations
day-02: practice linux filesystem
day-01: learn linux fundamentals
```

That is much more valuable than having 90 commits such as:

```
update
update2
final
final-final
test
test2
```

Final Repository

At Day 90, your repository should look approximately like this:

```
devops-journey/
├── README.md
├── progress.md
├── broke-it-fixed-it.md
├── 01-linux/
│   ├── setup-server.sh
│   ├── permissions-lab.md
│   └── ...
├── 02-bash/
│   ├── system-info.sh
│   ├── backup.sh
│   └── ...
├── 03-networking/
│   ├── networking-lab.md
│   ├── ssh-lab.md
│   └── troubleshooting.md
├── 04-git/
│   ├── git-workflow.md
│   └── git-recovery.md
├── 05-docker/
│   ├── Dockerfile
│   ├── docker-compose.yml
│   └── README.md
└──
```

```
├── 06-jenkins/
│   ├── Jenkinsfile
│   ├── docker-compose.yml
│   └── README.md
├── 07-github-actions/
│   ├── ci.yml
│   └── README.md
├── 08-kubernetes/
│   ├── deployment.yaml
│   ├── service.yaml
│   ├── configmap.yaml
│   ├── secret.yaml
│   └── README.md
├── 09-argocd/
│   ├── application.yaml
│   ├── helm-chart/
│   └── README.md
├── 10-ansible/
│   ├── inventory/
│   ├── playbook.yaml
│   ├── roles/
│   ├── templates/
│   └── README.md
├── 11-terraform/
│   ├── main.tf
│   ├── variables.tf
│   ├── outputs.tf
│   └── README.md
├── 12-aws/
│   ├── architecture.md
│   └── README.md
└── capstone/
    ├── README.md
    └── architecture.md
```

The Most Important Part

Don't treat this as 90 days of watching tutorials.

Use roughly this ratio every day:

100 minutes study session

```
30-35 min → Learn
60-65 min → Build / break / troubleshoot
10 min    → Document
10 min    → Commit
```